

Other Pre-Processor Commands

Additional preprocessor directives that enhance C programming



Conditional

`#ifndef, #if, #else, #elif`



Line Control

`#line number "string"`



Implementation

`#pragma directives`



Pre-defined

__DATE__, __TIME__,
__FILE__, etc.

1/5

Other Pre-Processor Commands

Conditional Commands

- **#ifndef** - If this macro is not defined
- **#if** - Test if a compile time condition is true
- **#else** - The alternative for #if. This is part of an #if preprocessor statement and works in the same way with #if that the regular C else does with the regular if.
- **#elif** - Enables us to establish an "if...else...if .." sequence for testing multiple conditions.

Line Control

#line number "string" – informs the preprocessor that the number is the next number of line of input. "string" is optional and names the next line of input. This is most often used with programs that translate other languages to C. For example, error messages produced by the C compiler can reference the file name and line numbers of the original source files instead of the intermediate C (translated) source files.

Pragma

#pragma is used to turn on or off certain features. Pragmas vary from compiler to compiler. Pragas available with Microsoft C compilers deals with formatting source listing and placing comments in the object file generated by the compiler. Pragas available with Turbo C compilers allows to write assembly language statements in C program.

A control line of the form

```
#pragma token-sequence
```

This causes the processor to perform an implementation-dependent action. An unrecognized pragma is ignored.



Pre-defined Names Defined by Pre-Processor

ANSI C defines a number of macros. Although each one is available for use in programming, the predefined macros should not be directly modified.



Pre-defined Macros

Macro	Description
<code>__DATE__</code>	The current date as a character literal in "MMM DD YYYY" format.
<code>__TIME__</code>	The current time as a character literal in "HH:MM:SS" format.
<code>__FILE__</code>	This contains the current filename as a string literal.
<code>__LINE__</code>	This contains the current line number as a decimal constant.

`__STDC__`

Defined as 1 when the compiler complies with the ANSI standard.



Example:

```
#include <stdio.h>

main()
{
    printf("File :%s\n", __FILE__ );
    printf("Date :%s\n", __DATE__ );
    printf("Time :%s\n", __TIME__ );
    printf("Line :%d\n", __LINE__ );
    printf("ANSI :%d\n", __STDC__ );
}
```

Output:

```
File :test.c
Date :Dec 04 2023
Time :13:26:14
Line :8
ANSI :1
```

Macros Vs Functions

We have discussed about macros, any computations that can be done on macros can also be done on functions. But there is a difference in implementations and in some cases it will be appropriate to use macros than function and vice versa.

Macros	Functions
Macro calls are replaced with macro expansions (meaning).	In function call, the control is passed to a function definition along with arguments, and definition is processed and value may be returned to call.
Macros run programs faster but increase the program size.	Functions make program size smaller and compact.
If macro is called 100 numbers of times, the size of the program will increase.	If function is called 100 numbers of times, the program size will not increase.
It is better to use Macros, when the definition is very small in size.	It is better to use functions, when the definition is bigger in size.



Key Decision Points

Use **macros** for small, frequently used operations where speed is critical and code size increase is acceptable.

Use **functions** for larger operations where code efficiency and maintainability are priorities.

4/5

Conclusion

The preprocessor makes programs easier to develop and modify. The preprocessor makes C code more portable between different machine architectures and customize the language.



Key Points About C Preprocessor

- The C Preprocessor is not part of the compiler, but is a separate step in the compilation process.
- All preprocessor lines begin with #.
- C Preprocessor is just a text substitution tool on your source code in three ways:



File Inclusion

Inserts the contents of another file into your source file.



Macro Substitution

Replaces instances of one piece of text with another.



Conditional Compilation

Arranges source code depending on various circumstances.



Final Thought

Mastering the C preprocessor is essential for writing efficient, portable, and maintainable code. Understanding when to use macros versus functions, and how to leverage conditional compilation, can significantly improve your C programming skills.

